

FEAST: A Framework for Evaluating Implementation Architectures of Self-adaptive IoT Systems

Federico Di Menna
University of L'Aquila
L'Aquila, Italy
federico.dimenna@student.univaq.it

Henry Muccini
University of L'Aquila
L'Aquila, Italy
henry.muccini@univaq.it

Karthik Vaidhyanathan
University of L'Aquila
L'Aquila, Italy
karthik.vaidhyanathan@univaq.it

ABSTRACT

Self-adaptation is considered as one of the effective means to enable IoT systems to overcome run-time uncertainties and thereby provide guarantees on the Quality of Service (QoS). Traditionally this is accomplished by extending the IoT system (managed system) with a managing system responsible for handling the adaptation. Many different approaches have been proposed in the literature to perform such adaptations but, they mainly focus on improving the adaptation logic inside the managing system to provide better guarantees on the QoS of the managed system. However, they do not focus on the implementation architecture of the managing system itself. Further, the choice of implementation can impact the QoS offered by the IoT system and that of the managing system. To this end, in this work, we propose a framework that allows the architect to systematically design, analyze and evaluate different architectural strategies for implementing the managing system of self-adaptive IoT systems. The evaluation results of our approach on the case study of a real-world IoT system demonstrate promising results in allowing architects to evaluate multiple architectural strategies for implementing the managing system.

ACM Reference Format:

Federico Di Menna, Henry Muccini, and Karthik Vaidhyanathan. 2021. FEAST: A Framework for Evaluating Implementation Architectures of Self-adaptive IoT Systems. In *Proceedings of ACM SAC Conference (SAC'22)*. Brno, Czech Republic, April 25-April 29, 2022 (SAC'22), 8 pages. <https://doi.org/10.1145/3477314.3507146>

1 INTRODUCTION

The last decade has seen self-adaptive software systems becoming increasingly popular primarily since software components are becoming more and more pervasive and therefore subjected to more uncertainties [23]. In addition, the goals/run-time requirements of the software keep changing. These uncertainties, if not handled properly, can have a significant impact on the system QoS. This holds true particularly in the case of data- and event-driven systems such as the IoT applications [22]. The emergence of IoT has revolutionized software development as it is about software and

systems containing software and a set of connected hardware elements (sensors, actuators, etc.) deployed in dynamic environments. On the one hand, IoT systems are becoming very popular, and they are widely adopted, whereas on the other hand, architecting and maintaining IoT systems poses many challenges arising from heterogeneity, interoperability, security, etc. [1, 11]. These challenges coupled with run-time uncertainties can have a significant impact on the overall QoS offered by the system [1, 22].

One potential solution to enable the IoT systems to overcome run-time uncertainties is to architect IoT systems with the ability to self-adapt their architecture/configurations to handle the uncertainties. Based on the conceptual model of a self-adaptive system [23], any self-adaptive IoT system consists of: i) a managed system, which represents the running IoT system; ii) goals that define the set of requirements the IoT system needs to accomplish (e.g., sensors of the IoT system should not consume more than 10.0 joules every 10 minutes); iii) managing system, that handles the adaptation logic driven by the goals to adapt the architecture/configuration of the IoT system; and iv) the environment with which the IoT system interacts or within which the system is placed/deployed. Traditionally, in an architecture-based self-adaptive system, the managing system is accomplished using the MAPE-K framework [16] which consists of four consecutive activities or phases, namely, Monitor, Analyze, Plan, and Execute with a Knowledge that is shared across the different activities. These activities are together responsible for handling the adaptation of the IoT system.

Different approaches to handling self-adaptation of IoT systems have been proposed in literature [14, 20]. These include both proactive and reactive approaches, focusing on different goals and using various techniques (ranging from rule-based methods to quantitative verification, machine learning, etc.) to handle the adaptation logic inside the MAPE-K. These approaches focus more on how each activity of the MAPE-K work but, they do not essentially focus on the technical implementation architecture of the MAPE-K itself [10]. For example, the component responsible for monitoring can interact with the component responsible for analyzing using a simple socket connection or invoke the analysis component using a remote call. Similarly, each component could be implemented using different programming languages. Each such choice can impact the QoS of the managing system itself and further impact the effectiveness and the efficiency of the adaptation performed.

To this end, in this work, we propose FEAST, design analysis and evaluation framework that allows architects/engineers to consider different architectural strategies (AS) for implementing the managing system and further analyze and evaluate the strategies with respect to the adaptation goals of the IoT system as well as the QoS goals of the managing system. Hence, the main contribution of the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
SAC '22, April 25–29, 2022, Virtual Event,
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-8713-2/22/04...\$15.00
<https://doi.org/10.1145/3477314.3507146>

work is a framework that provides architects or self-adaptation engineers with abilities to i) define different application scenarios of the IoT system; ii) consider different technical alternatives for implementing the managing system in the form of AS; iii) define the adaptation goals of the IoT system and QoS goals of the managing system; iv) simulate the different AS; v) analyze and evaluate the AS with respect to the goals and vi) identify the optimal AS for implementing the managing system. Further, we demonstrate the effectiveness of our framework by applying it to the case study of a real-world IoT system. The results of our evaluation show the impact of different AS on the adaptation goals of the IoT system, thereby emphasizing the advantage offered by the framework to architects in identifying an optimal AS for implementing self-adaptive IoT systems.

2 MOTIVATING SCENARIO

We illustrate our approach on a scenario that concerns the NdR street fair, which is a scientific exhibition event organized by the University of L'Aquila¹. In this event, the research community and public (about 25,000 visitors) are brought together to participate in activities held in the city center, in which performances, demonstrations, and workshops take place in squares, main streets, and buildings.

Our research group has developed an IoT-based solution to improve the quality of the visiting experience. Without the loss of generality, in this paper we focus on two such IoT services planned for NdR. These services are related to the automated management of three venues (with capacity for 500 and 2×200 seats) and control of two automated parking lots (capacity 100 and 75 slots). These parking lots and venues are created in an ad-hoc way and on a temporary basis. Hence, availability of external power sources and legacy infrastructure is limited.

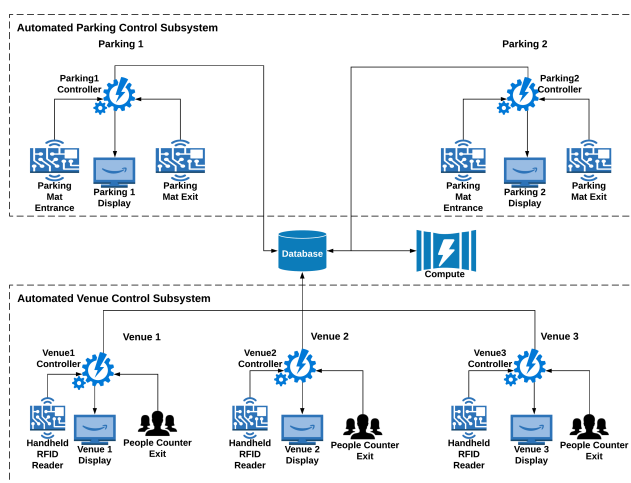


Figure 1: High level view of the NdR system

Figure 1 shows the overall architecture of the system. It uses sensors at the venue and parking entrances and exits to provide real-time

¹<http://streetscience.it>

information on the venue and parking lots availability so as to redirect the users to different parking lots based on their venue preferences. Each of these sensors can operate in: (i) *normal mode*, in which the sensor component gathers data at standard rate (normal operational scenario, and (ii) *critical mode*, in which the sensor component gathers data with higher frequency (e.g., when there is a sudden increase in people/vehicle arrival/departure rate, the data needs to be gathered at a faster rate to ensure service accuracy). However, one of the constraints we have is to ensure that the sensors, such as the parking mats and the venue counters, being battery-powered, need to operate in an energy-efficient manner as they can run only for half a day. Even though charging can be performed, it might happen that the charge goes down during peak hours, and plugging in a new sensor/charging the existing one can be costly in terms of crowd management. To this end, we added a managing system on top of the NdR IoT system that continuously monitors the QoS (energy) and application data and performs proactive adaptation using machine learning techniques to save the battery. However, NdR being a data-intensive IoT system, the velocity of data is high. Hence, the implementation of the managing system needs to be effective and efficient to incorporate such velocity and further reduce the energy consumed by the IoT devices.

Hence, given an IoT system with a set of QoS requirements (energy in this scenario), the goal of this work is to develop a framework that allows architects to define an optimal architecture for implementing the managing system by: i) eliciting the different possible application scenarios; ii) defining the QoS goals of the managing system (response time or the total time taken to perform the adaptation in this scenario); and iii) considering different AS for implementing the managing system; iv) analyzing and evaluating each of the AS to select an optimal strategy that provides overall guarantees on the QoS goals of the managed and managing system.

3 THE FEAST FRAMEWORK

This section presents the FEAST framework that allows architects or self-adaptation engineers to define, analyze and evaluate different AS for the implementation of the managing system. Figure 2 represents the proposed FEAST framework. It is based on the Cost-Benefit Analysis Method (CBAM) for evaluating architectural alternatives [3, 15]. It primarily consists of four main stages: i) the *Define Scenarios* stage that allows architects to define the different operational scenarios of the managing system; ii) the *Define Architectural Strategies* stage allows architects to design the system by generating different architectural strategies for the implementation of the managing system for each defined scenarios; iii) the *Analysis* stage that simulates the managed system and applies cost-benefit approach for analyzing the different architectural strategies; and, iv) the *Evaluation* stage that evaluates the results of the analysis and identifies the optimal architectural strategy for the implementation of the managing system. It also provides a detailed evaluation of the different architectural alternatives with respect to the cost/benefit trade-offs. In the following section, we thoroughly explain each of the four steps depicted in Figure 2.

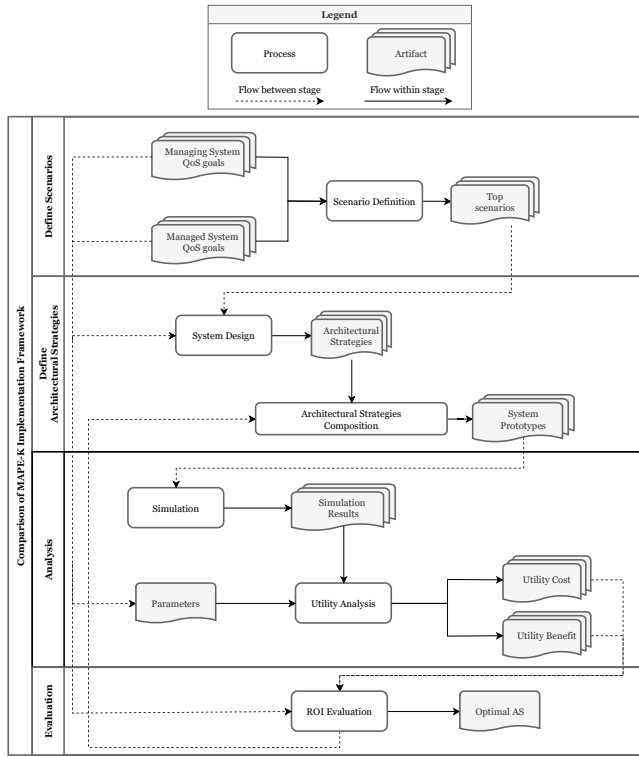


Figure 2: The FEAST Framework

3.1 Define Scenarios

This represents the first stage where the architect/engineers define a set of operational scenarios of the managing system. It mainly consists of the *Scenario Definition Process*. In this process, the architect is responsible for eliciting the different scenarios in which there is a control/data flow from the managed system to the managing system. For example: in the case of NdR case study (refer Section 2), one of the possible scenarios is that the managing system needs to gather the QoS and/or application data from the NdR IoT system every 1 second especially in the late hours as it will be more crowded to make accurate adaptation decisions. On the other hand, in a non-crowded scenario, a managing system would need to monitor the IoT system every 1 minute or 10 minutes. All these possible operational scenarios will be defined during this process. Further, the scenario definition process takes two key inputs:

1. *QoS goals of the managed system*, for each scenario, for different QoS metrics of the IoT system are defined by the respective stakeholders through a configuration file. For example a QoS goal, $Q_{S1} = (energy, 100.0, joules)$ for a period $t = 10 \text{ minutes}$, states that the total energy consumed by the system for 10 minutes under scenario 1 should not exceed a maximum value 100.0 joules.
2. *QoS goals of the managing system*, for each scenario, for different QoS metrics of the components of the managing system or the managing system as a whole are defined by the self-adaptation engineers/architects through a configuration file. For example a QoS goal, $Q_{MS1} = (RT, 0.05, seconds, analyze)$, states that the average response time of the analyze activity in Scenario 2 should

not be more than 0.05 seconds. Similarly, a QoS goal, $Q_{MS2} = (RT, 0.6, seconds, full)$ states that the average response time of the entire MAPE-K loop ie, the total time taken to perform an adaptation in Scenario 2 should not be more than 0.6 seconds.

This stage then produces as output a set of *Top scenarios* which represent the important or critical scenarios selected by the architects after considering the use-case, the problem domain, requirements, etc. These scenarios are used in the next stage of the framework.

3.2 Define Architectural Strategies

This stage allows the architects/engineers to come up with AS, which could be used for implementing the managing system for each of the scenarios generated in the previous stage. It consists of two main processes:

System Design. The respective stakeholders use this process to generate the different possible system designs for implementing the managing system. It takes as input the main scenarios and the associated QoS goals. It then uses these inputs to generate different *Architectural strategies* as output, which then serves as candidates for the implementation of the managing system. Generating AS implies defining components of the managing system, their connectors, and the corresponding behavior. The managing system is typically implemented based on the MAPE-K loop. Four different components may perform these activities. Now each of the components can be implemented using different technologies, and further, they may be connected with each other using different mechanisms. The technological choice for component implementation, type of connectors, and associated behavior result in different types of AS. In this work, we use three different base AS for each of the scenarios resulting from the first stage. These strategies can be further expanded using different implementation technologies to create more AS for implementation. The three base strategies include:

- i) **Centralized Connector.** In this strategy, there will be a central component with which all components of the managing system communicate (refer Figure 3). This central component also serves as the Knowledge base. In this case, therefore, the centralized connector will be incorporated into the Knowledge. The centralized can be implemented in different ways. For example, through an in-house database, a cloud database, or even a centralized file system.

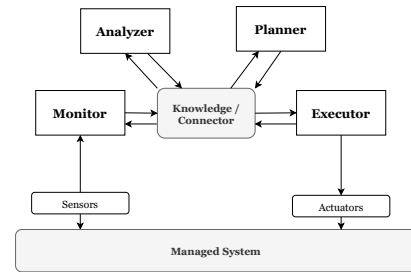


Figure 3: Centralized connector

- ii) **Publish Subscribe Connector.** In this case (refer Figure 4), connectors in the form of topics will be used for communication among different components of the managing system. As we can see in the figure 4, there is a topic that allows unidirectional communication

between Monitor and Analyzer, a topic for the transferring the data from the Analyzer to the Planner, and another from the Planner to the Executor. As opposed to a Centralized connector, in this case, the four MAPE components do not necessarily communicate using the Knowledge component. The communication is split into two parts: i) a data flow for feedback, and; ii) another between the components and the Knowledge. Since we are using Publish-Subscribe connectors, it is essential to underline that the communication between each MAPE activity and the Knowledge will be managed by topic to allow reading/writing on the Knowledge. The architecture and the number of these topics will change according to the implementation strategies. For instance, just one topic could be used for all communication between the MAPE components, or one topic per component could also be employed.

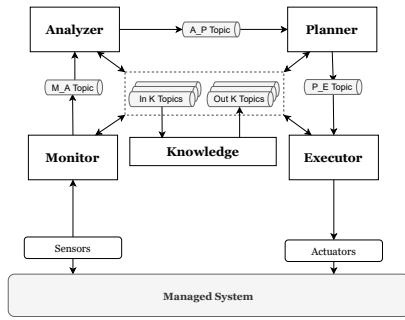


Figure 4: Publish Subscribe connector

iii) **Socket Connector**. In this case, the MAPE components are all connected through TCP-based socket connections. The flow is similar to the previous one. We split the data flow in two: one for the feedback loop and one for communicating with the centralized knowledge. The difference, however, is that this time the communication between all components is achieved through a TCP connection using sockets. In this case, all communication will be bidirectional.

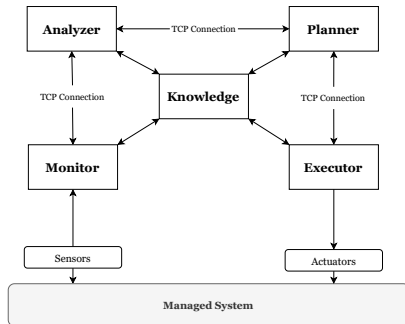


Figure 5: Socket based connector

Architectural Strategies Composition. This process takes as input the different AS of the managing system. Each of the strategies is then integrated with the IoT system to generate the working prototype of the IoT system. For each AS, a system prototype is generated, and this is then used as the input for the Analysis stage.

3.3 Analysis

This stage forms the crucial stage of the FEAST framework (Figure 2). It allows the architects to analyze the different AS with respect to the QoS goals of the managed system and that of the managing system. This stage comprises two main processes: i) **Simulation** and ii) **Utility Analysis**. They are described below.

Simulation. This process is responsible for simulating the IoT system (managed system) of each of the system prototypes generated by the previous stage. Architects can then make use of the results of the simulations and the execution logs of the managing system to analyze the quality aspects of the different architectural strategies. Such a simulation of the IoT system can be performed using different tools and techniques. In this work, we make use of CupCarbon IoT simulator [4] for simulating the IoT system. It is a state-of-the-art simulator especially for simulating the energy consumption of IoT devices [17]. This process internally consists of two main steps: i) **Simulation setup**, in which we convert the system prototype(s) into a simulatable IoT system(s). This can be achieved with the help of modeling tools like CAPS [19] or ThingML [12] which supports the modeling of IoT systems. Further, the modeled system could be simulated using tools like CupCarbon, which we have used in this work. During this step, we define the configuration of the IoT devices, frequency of communication, type of processing node, memory, etc. Further, we also define the configuration of the managing system components, such as the frequency of monitoring, configuration of the connectors in the managing system (e.g., number of topics if it is a publish-subscribe connector, timeout for socket connector, etc.).

ii) **Simulation execution**, in which the IoT system is simulated, and the results are collected. During this step, we first define the time for which the simulation needs to be executed. The simulation will result in a set of *simulation results* which consists of three main outputs: i) energy consumption logs, which indicate the energy consumed by the sensors during the simulation; ii) IoT system execution logs, which contains the information on the data exchanged by the IoT devices during the simulation; and iii) Managing system execution logs, which contains the information on the QoS of the different components of the managing system such as the response time, CPU time, number of adaptations performed, etc. In this work, we use the log files produced by CupCarbon to serve as the sensing interface to the managing system. Further, the actuation interface for executing the adaptations is achieved through a configuration file.

The results of the simulation are then passed to the Utility Analysis process as input for further processing and analysis.

Utility Analysis. This process takes as input the simulation results and the parameters for analysis. It then performs the following:

i) **Aggregation:** The logs generated may not be uniform with respect to the time intervals t specified in the QoS goals of the managed system and managing system. Towards this, data aggregation is performed to convert the energy consumption logs of the IoT system (of each AS) into a uniform time interval-based energy consumption dataset, E_a , by aggregating the data into t minute intervals. The same step is applied to the execution logs of the managing system to generate an aggregated response time dataset, R_a .

ii) *Normalization*. Each of the datasets may follow a different scale. For instance the energy consumption dataset of an AS, a_1 represented by E_{a1} may have a range between $[15.0, 30.0]$ for a time t , whereas, response time dataset R_{a1} may have a range between $[0.04, 0.20]$. Further, the range of every E_a/R_a may differ for each $a \in AS$. To this end, we apply normalization to ensure uniformity in the scale of data across the energy consumption and response time datasets generated for each of the $a \in AS$. For instance, let $E_{high} = \max(E(e_a, t) \forall a \in AS)$, represent the maximum energy consumed in given time interval t , let $E_{low} = \min(E(e_a, t) \forall a \in AS)$ represent the value of the minimum energy consumed for a time interval t and let l represent the total number of aggregated time intervals. Then normalized energy dataset ne_a for each $a \in AS$ is given by the set, $\{ne1_a, ne2_a \dots ne_l_a\}$ where, $nei_a = (ei_a - E_{low}) / (E_{high} - E_{low}) \forall i \in l$. The same process is repeated for response time datasets. This process normalizes the entire datasets to the range $[0, 1]$.

iii) *Cost Calculation*. This step allows the architect to analyze the total utility in terms of the costs incurred for the implementation of a given AS. This cost can be the economic cost that has to be incurred for implementing a particular AS or the cost in terms of the QoS of the components of the managing system with respect to the QoS goals of the managing system. This cost is then used for evaluation of the different AS in the next stage. It follows the CBAM approach of calculating the expected cost. However, while CBAM uses a manual analysis, we use an automated approach using a cost-utility function. This is applied to the QoS values extracted from the execution logs of the managing system. In this work, we want the managing system to be highly efficient in executing the adaptation. Hence, we consider the total response time of the MAPE components of the managing system as the cost. Let R_{max} denote the response time threshold for the managing system, the total cost, C incurred by the managing system for a given architectural strategy a for an adaptation interval, τ (e.g., every 1 minute, 10 minute, etc.), is obtained by computing the cost-utility:

$$C_{a\tau} = (1 - R_\tau); R_\tau = \begin{cases} R_{max} - r_\tau & \text{if } r_\tau < R_{max} \\ (R_{max} - r_\tau) \cdot p_r & \text{otherwise} \end{cases} \quad (1)$$

where, r_τ represent the average response time of the managing system the τ -period, and R_τ is a piece-wise function that capture the response time savings where, $p_r \in \mathbb{R}^+$ represent penalties for the violations of response time thresholds. It is important to note that the cost, at every τ interval, is computed by subtracting 1 from the value of the piece-wise function, R_τ . This is done to ensure that the cost is small when the value of R_τ is higher which essentially means that the managing system is able to provide higher guarantees on the threshold, R_{max} .

iv) *Benefit Calculation*. While the previous step calculates the cost associated with the managing system, this step focuses on calculating the benefit of implementing a given AS on the IoT system (managed system). This step also takes an automated approach similar to the previous step by calculating the benefit using a benefit utility function. This function is applied to the QoS values obtained from the logs of the IoT system. In this work, we want to reduce the energy consumed by the IoT system. Hence we consider the energy saved by the IoT system as the benefit. Let E_{max} denote the

maximum energy that can be consumed by the IoT system (threshold) for a given time interval τ , then total benefit, B accrued by the IoT system while using a given architectural strategy a for the implementation of the managing system, is obtained by computing the benefit utility:

$$B_{a\tau} = E_\tau; E_\tau = \begin{cases} E_{max} - e_\tau & \text{if } e_\tau < E_{max} \\ (E_{max} - e_\tau) \cdot p_e & \text{otherwise} \end{cases} \quad (2)$$

where, e_τ represent the energy consumed by the IoT system for the period, τ , and E_τ is a piece-wise function that capture the energy savings where, $p_e \in \mathbb{R}^+$ represent penalties for the violations of energy thresholds.

The utility functions can be extended easily to incorporate different and multiple QoS parameters of the managing and IoT system. The values for $R_{max}, p_r, E_{max}, p_e$ can be passed by the respective stakeholders as input parameters to the Utility Analysis process.

3.4 Evaluation

This stage forms the final stage of the approach. It is responsible for evaluating different AS for implementing the managing system for each of the scenarios. It consists of the *ROI Evaluation* process, which identifies the optimal AS by considering the QoS goals of the managed system and that of the managing system. It takes as input the cost and benefit values obtained from the Analysis stage. Further, as proposed by the CBAM framework, it calculates the Return on Investment (ROI) provided by each of the AS. This process allows us to compute the ratio of the cost incurred by the managing system when using an AS with respect to the benefit provided by the implementation of an AS on the IoT system. The ROI provided by each of the architectural strategies, $a \in AS$ for a scenario, is given by the formula:

$$R_{Ia} = \sum_{\tau=1}^n \frac{B_{a\tau}}{C_{a\tau}} \quad (3)$$

where, $\tau = \{1, \dots, n\}$ represents the n adaptation intervals, $B_{a\tau}$ and $C_{a\tau}$ represents the benefit and cost obtained for each τ while using an AS, a respectively. For each scenario, this process then ranks the different AS based on their corresponding ROI values. The AS that provides the highest ROI can be used for implementing the managing system for a given scenario.

In addition, the framework provides flexibility for the architects to easily refine or modify the AS by going back to stage two of the framework or can modify the QoS goals, redefine the parameters, etc and re-run the analysis and evaluation stages to gather results. In this manner, architects can quickly and systematically evaluate different AS for the implementation of the managing system.

4 EXPERIMENTS AND EVALUATIONS

In this Section, we provide details on how we incorporated the FEAST framework for selecting the optimal AS for implementing the managing system of the NdR IoT system (refer Section 2).

4.1 Experiment Setup

The complete experiment was performed on a Microsoft Azure virtual machine running Windows Server 2019 Datacenter OS with 32 GB RAM and 4 vCPUs. The IoT system of the NdR case study was realized using the CupCarbon IoT simulator. The managed and the managing system were deployed on this virtual machine.

4.2 Experiment candidates

To identify the optimal AS for implementing the managing system of the NdR IoT system, we considered two scenarios in stage 1 of the FEAST framework. For each scenario, we considered three different AS. The details on the scenarios and AS are provided below.

Candidate scenarios. After analyzing the different possible operational scenarios of the NdR case study, We took into consideration two top scenarios (refer Section 3). These scenarios were based on two parameters: i) *simulation time*: This is the time for which the IoT system is simulated; ii) *adaptation cycle interval*: This is the interval in which the managing system collects the data from the IoT system for performing the activities of the MAPE-K loop.

Scenario 1. This scenario represents the regular hours of the NdR fair, where people come at a lower rate during the afternoon and at a higher rate during the later hours. During this period, the managing system needs to collect data at regular intervals from the NdR system to perform effective adaptations. The simulation of this scenario was performed using the data based on the real observations of the NdR event. The parameters used are: i) simulation time: 86400.0 s; and ii) adaptation cycle interval: 60.0 s

Scenario 2. This scenario represents the NdR fair's two peak hours in which a large number of the crowd move into and out of the venues. During this period, the managing system needs to collect data at a high frequency from the NdR IoT system to ensure that the adaptation performed is effective and efficient with respect to the high rate of inflow of data captured by the IoT system. Similar to scenario 1, the simulation was performed using the real observations of the NdR case study. The parameters used are: i) simulation time: 7200.0 s; and ii) adaptation cycle interval: 1.0 s

Candidate Architectural Strategies. We used the three AS presented in Section 3 as the candidate strategies for each of the two scenarios. In order to implement the different strategies of the system, we used different technologies. Python (version 3.8) was used to implement the components of the managing system (Monitor, Analyzer, Planner, Executor). For the AS that uses *Socket connector*, the TCP connection was established using native Python. Instead, for the AS that uses the *Centralized Connector*, we relied on the Blackboard Architectural Style. There are three general components in a blackboard style: the Knowledge Source(s), the Blackboard, and the Controller. The *Knowledge Source* component, in this case, is only one, and it involves the sensor network. The Blackboard component is the data container, and all the components will use this one to read and write data. We used MySQL Server² to implement the data storage in the Blackboard, in particular, a single database with a table for each component. Each MAPE component stores the data in the database (Knowledge), which the next one reads. Finally, for the AS that uses *Publish Subscribe connector*, the communication

among the modules is realized through event-driven mechanisms. More precisely, we made use of Apache Kafka³ for the implementation of the message broker with a replication factor 1 and three topics. The three topics were used for the communication between the different components of the managing system. In every AS, the Analyzer component used a pre-trained machine learning model (with high accuracy) to predict the expected energy consumption of the IoT system for the next 10 minutes, and further, the planner used a rule-based adaptation logic that adapts the communication frequency of the sensor components to reduce energy consumption. The complete details of the implementation, along with source code and models used, can be found here⁴.

4.3 Evaluation Metrics

We used the following metrics for identifying the optimal AS for the two scenarios: i) *The average response time (ART)* of the MAPE-K loop, which denotes the average time taken by all the components of the managing system to complete one adaptation cycle; ii) *Total energy consumed (E)* by the IoT system during the entire execution of the simulation; iii) *Cumulative Utility Cost (C)*, iv) *Cumulative Utility Benefit (B)*; and v) *Cumulative Return on Investment, (ROI)* where C , B and ROI denotes the accumulated values of the cost-utility function (refer Equation 1), benefit utility function (refer Equation 2), and the ROI (refer Equation 3) respectively over the entire duration of the execution.

4.4 Results

This subsection describes the results obtained during the analysis and evaluation process after simulating the IoT system with different AS. To this end, we defined scenario-specific goals for the energy consumption of the IoT system and the total response time of the managing system. These goals were defined in the form of thresholds E_{max} and R_{max} denoting the energy and response time thresholds, respectively, as explained in Section 3 (These goals can easily be configured. What used here is mainly to demonstrate the capability of our framework). The results obtained during simulation in the form of execution logs, where analyzed using Python scripts that implement the *Utility Analysis process* (Stage 3 of the FEAST framework, refer Section 3). These results were then used to perform the *ROI evaluation process* implemented using Python (Stage 4 of the FEAST framework) to identify the optimal AS for each scenario. The results obtained during the analysis and evaluation of AS for each scenario are described below.

Scenario 1 Figure 6 shows the plot of the cumulative ROI accrued while using each of the three AS in scenario 1 for goals: i) $R_{max} = 0.06$ seconds and ii) $E_{max} = 15.0$ joules. We can notice that the maximum value is reached by the AS that uses a centralized connector. Detailed results are depicted in Table 1. For the thresholds specified, the ROI offered by the AS, centralized connector (187.27) is just 2% higher than that of the one that uses a socket connector (185.29). This is because, in the centralized connector, the efficiency of reads and writes provided by MySQL allows the managing system components to quickly communicate with each other. Contrary to the popular assumption, publish-subscribe does not provide a

²<https://dev.mysql.com/>

³<https://kafka.apache.org/>

⁴<https://github.com/federix98/FEAST>

Scenarios	AS	Thresholds		Avg RT	E	C	B	ROI
		R_{max}	E_{max}					
Scenario 1	Socket	0.06	15	0.026	2083.383	45.708	60.665	185.89
		0.08	14			44.761	54.905	171.80
		0.05	18			46.181	77.945	236.39
	Centralized	0.06	15	0.064	2009.429	47.525	63.623	187.27
		0.08	14			46.579	57.863	173.76
		0.05	18			47.999	80.903	235.86
	Publish Subscribe	0.06	15	0.062	2079.661	47.108	60.814	178.84
		0.08	14			46.168	55.054	165.24
		0.05	18			47.578	78.094	227.26
Scenario 2	Socket	0.06	15	0.02	70.958	3.839	9.162	28.64
		0.08	14			3.759	8.682	27.71
		0.05	18			3.879	10.602	32.80
	Centralized	0.06	15	0.035	86.893	3.9	8.524	26.23
		0.08	14			3.82	8.044	25.27
		0.05	18			3.94	9.964	30.35
	Publish Subscribe	0.06	15	0.04	85.687	3.92	8.573	26.24
		0.08	14			3.84	8.093	25.28
		0.05	18			3.96	10.013	30.33

Table 1: Results of analysis and evaluation of different AS for each of the two scenarios (Avg RT in ms and E in joules)

higher ROI. This is because the time required to push an event to a topic and further consume is much higher than that needed for the communication using sockets or a centralized database. It is also worth noticing that the energy consumed by the IoT system when using the managing system implemented by AS, the centralized connector is almost 70 joules less than that consumed by the other AS whereas the $Avg.RT$ is higher than that offered by the other AS. On the other hand, the $Avg.RT$ is the minimum when using the AS, Socket connector, but it leads to the maximum energy consumption of the IoT system. Since the benefit (with respect to energy consumption) is high, in the optimal AS for thresholds, $R_{max} = 0.06$ seconds and $E_{max} = 15.0$ joules is the strategy based on centralized connector. Table 1 also shows the results of different metrics when using different thresholds for R_{max} and E_{max} . For instance, we can observe that when $R_{max} = 0.05$ s and $E_{max} = 18.0$ joules, the AS that uses socket connector offers the highest ROI. Similarly, an architect can make use of different threshold values to identify the optimal AS. From the results for scenario 1, we can hence conclude that the AS centralized connector offers the maximum ROI and hence is the optimal AS to be considered for implementation of the managing system under scenario 1.

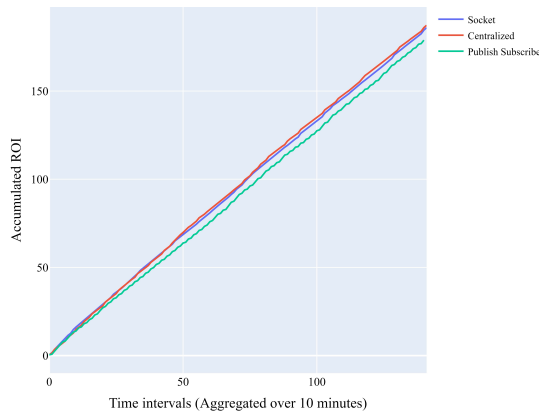


Figure 6: Scenario 1 - accumulated Return On Investment

Scenario 2 Figure 7 shows the cumulative ROI of scenario 2 while using each of the three AS. The first set of goals used were the same as that of the ones used in Scenario 1. Different from Scenario 1, in this scenario, we can observe that the AS that uses the Socket connector provides the highest ROI. Initially, we can observe that all three offered similar ROI. However, as time progresses, we can see that the gap between the AS of socket connector and others starts increasing. Table 1 (Scenario 2) shows the detailed results. It can be observed that the energy consumed by the IoT system when using the AS socket connector for the implementation of the managing system is about 15 joules less than the energy savings offered by other approaches. Also, the average time taken to perform the entire adaptation cycle in the case of socket connector is the least while compared to other AS. The same trend holds true for the different sets of energy and response time thresholds as well. This shows that in a more data-intensive scenario where the managing system needs to monitor the IoT system at a much higher rate, the AS that uses socket connection is the optimal strategy.

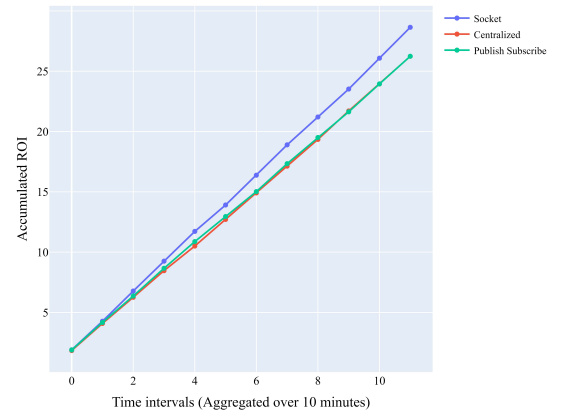


Figure 7: Scenario 2 - accumulated Return On Investment

4.5 Threats to Validity

Threats to *construct validity* relate to the use of different simulation and goal parameters. To this end, we used configurations and parameters based on the actual observations from the NdR case study. Threats to *external validity* concern the generalizability and scalability of our approach. Although our approach has been applied to the NdR IoT system, it uses techniques that can be generalized to a more complex class of IoT systems or other software systems in general as long as there is a means to simulate the managed system and collect the logs of the managing system.

5 RELATED WORKS

Different approaches to self-adaptation in IoT/CPS has been proposed in literature [20]. Iftikhar et al. in [14] proposed an approach based on the ActivForms environment that uses statistical model checking to perform adaptation in IoT systems. Weyns et al. in [24] propose an approach for managing run-time uncertainties associated with over-provisioning of resources in IoT systems. Fahad et al. [2] propose the notion of *emergent configuration* for engineering self-adaptive goal-oriented IoT systems. There has also been

different Model Driven Engineering (MDE) based approaches such as [6, 7, 21] that use the concept of *models@run.time* and MAPE-K loops for adaptation. Nascimento et al. [8] propose an agent-based framework for performing self-adaptation for IoT applications. Cámara et al. [5] developed an approach that uses a combination of machine learning and quantitative verification techniques to perform architectural adaptation of IoT systems. Each these approaches implement MAPE-K using different technologies and mainly focus on improving/introducing techniques for improving the adaptation logic in the managing system and not on the architecture of the managing system itself. There has also been different self-adaptive exemplars that has been proposed in literature. This includes exemplars for evaluating self-adaptive solutions for IoT [13], unmanned underwater vehicles [9], web applications [18], etc. However, each of these exemplars allows architects/engineers to develop/evaluate self-adaptation techniques for the respective domains and does not provide mechanisms to evaluate the implementation architecture of the managing system itself.

Differently to the existing approaches, which focus mainly on developing novel logic/approaches for improving the adaptation performed by the managing system [10], in this work, we shift the focus by developing a framework that allows the architect/engineers to design and evaluate the implementation architecture of the managing system. Further, it also allows measuring the effectiveness of the architecture with respect to the goals of the IoT system (managed system) and that of the managing system.

6 CONCLUSION

In this work, we presented FEAST, a framework that allows architects or self-adaptation engineers to systematically define different operational scenarios of the IoT system, define QoS goals of the IoT system and the managing system, design architectural strategies for implementation, analyze the strategies, and finally evaluate the strategies in terms of the cost and benefit provided, to identify the optimal implementation strategy. The results of our evaluations on a real-world IoT case study demonstrate that our approach can allow architects to identify the optimal architectural strategy for implementation. To this end, we used two scenarios, each with three architectural strategies. The results further emphasized the need for such a framework as depending on the type of scenario and QoS goals, different architectural strategies were found to perform better. Moreover, the evaluations also show how the architects can use the FEAST framework to define different QOS goals, modify analysis parameters, and re-run evaluations to test different architectural strategies to implement the adaptation loop.

With respect to future work, the first research avenue we plan to explore is generalizability. To this end, we will be applying our approach to different classes of software systems and self-adaptive exemplar systems (e.g., DelatIoT, SWIM, etc.). The second extension of our approach is to incorporate sustainability framework(s) to identify optimal architectural strategy by considering the overall sustainability goals. Finally, we plan to evolve this approach into a tool that can be used by software architects to better design and implement self-adaptive software systems.

REFERENCES

- [1] Zainab H Ali, Hesham A Ali, and Mahmoud M Badawy. 2015. Internet of Things (IoT): Definitions, Challenges and Recent Research Directions. *International Journal of Computer Applications (0975-8887) Volume* (2015).
- [2] Fahed Alkhabbas, Romina Spalazzese, and Paul Davidsson. 2017. Architecting emergent configurations in the internet of things. In *2017 IEEE International Conference on Software Architecture (ICSA)*. IEEE, 221–224.
- [3] Len Bass, Paul Clements, and Rick Kazman. 2003. *Software architecture in practice*. Addison-Wesley Professional.
- [4] Ahcène Bounceur. 2016. CupCarbon: A New Platform for Designing and Simulating Smart-City and IoT Wireless Sensor Networks (SCI-WSN). In *Proceedings of the International Conference on Internet of Things and Cloud Computing (ICC '16)*. ACM, New York, NY, USA, Article 1, 1 pages.
- [5] Javier Cámara, Henry Muccini, and Karthik Vaidhyanathan. 2020. Quantitative verification-aided machine learning: A tandem approach for architecting self-adaptive iot systems. In *2020 IEEE International Conference on Software Architecture (ICSA)*. IEEE, 11–22.
- [6] Federico Ciccozzi and Romina Spalazzese. 2016. MDE4IoT: supporting the internet of things with model-driven engineering. In *International Symposium on Intelligent and Distributed Computing*. Springer, 67–76.
- [7] Mirko D'Angelo, Annalisa Napolitano, and Mauro Caporuscio. 2018. CyPhEF: a model-driven engineering framework for self-adaptive cyber-physical systems. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*. ACM, 101–104.
- [8] Nathalia Moraes do Nascimento and Carlos José Pereira de Lucena. 2017. Fiot: An agent-based framework for self-adaptive and self-organizing applications based on the internet of things. *Information Sciences* 378 (2017), 161–176.
- [9] Simos Gerasimou, Radu Calinescu, Stepan Shevtsov, and Danny Weyns. 2017. UNDERSEA: an exemplar for engineering self-adaptive unmanned underwater vehicles. In *2017 IEEE/ACM 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE, 83–89.
- [10] Ilias Gerostathopoulos, Thomas Vogel, Danny Weyns, and Patricia Lago. 2021. How do we Evaluate Self-adaptive Software Systems?: A Ten-Year Perspective of SEAMS. In *2021 International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. 59–70.
- [11] Sarra Hammoudi, Zibouda Aliouat, and Saad Harous. 2018. Challenges and Research Directions for Internet of Things. *Telecommun. Syst.* 67, 2 (Feb. 2018), 367–385. <https://doi.org/10.1007/s11235-017-0343-y>
- [12] Nicolas Harrand, Franck Fleurey, Brice Morin, and Knut Eilif Husa. 2016. ThingML: a language and code generation framework for heterogeneous targets. In *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems*. 125–135.
- [13] M Usman Iftikhar, Gowri Sankar Ramachandran, Pablo Bollansée, Danny Weyns, and Danny Hughes. 2017. DeltaIoT: a self-adaptive internet of things exemplar. In *Proceedings of the 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. IEEE Press, 76–82.
- [14] M Usman Iftikhar and Danny Weyns. 2017. Activforms: A runtime environment for architecture-based adaptation with guarantees. In *2017 IEEE International Conference on Software Architecture Workshops (ICSAW)*. IEEE, 278–281.
- [15] Rick Kazman, Jai Asundi, and Mark Klien. 2002. *Making architecture design decisions: An economic approach*. Technical Report. CARNEGIE-MELLON UNIV PITTSBURGH PA SOFTWARE ENGINEERING INST.
- [16] J. O. Kephart and D. M. Chess. 2003. The vision of autonomic computing. *Computer* 36, 1 (Jan 2003), 41–50. <https://doi.org/10.1109/MC.2003.1160055>
- [17] Cristina Lopez-Pavon, Sandra Sendra, Juan F Valenzuela-Valdés, et al. 2018. Evaluation of CupCarbon Network Simulator for Wireless Sensor Networks. *Netw. Protoc. Algorithms* 10, 2 (2018), 1–27.
- [18] Gabriel A Moreno, Bradley Schmerl, and David Garlan. 2018. SWIM: an exemplar for evaluation and comparison of self-adaptation approaches for web applications. In *2018 IEEE/ACM 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE, 137–143.
- [19] H. Muccini and M. Sharaf. 2017. CAPS: Architecture Description of Situational Aware Cyber Physical Systems. In *2017 IEEE International Conference on Software Architecture (ICSA)*. 211–220. <https://doi.org/10.1109/ICSA.2017.21>
- [20] Henry Muccini, Mohammad Sharaf, and Danny Weyns. 2016. Self-adaptation for cyber-physical systems: a systematic literature review. In *Proceedings of the 11th international symposium on software engineering for adaptive and self-managing systems*. 75–81.
- [21] Francisco Javier Acosta Padilla. 2016. *Self-adaptation for Internet of things applications*. Ph.D. Dissertation.
- [22] A. Taivalsaari and T. Mikkonen. 2017. A Roadmap to the Programmable World: Software Challenges in the IoT Era. *IEEE Software* 34, 1 (Jan 2017), 72–80.
- [23] Danny Weyns. 2020. *An Introduction to Self-adaptive Systems: A Contemporary Software Engineering Perspective*. John Wiley & Sons.
- [24] Danny Weyns, M Usman Iftikhar, Danny Hughes, and Nelson Matthys. 2018. Applying architecture-based adaptation to automate the management of internet-of-things. In *European Conference on Software Architecture*. Springer, 49–67.